Ethernet-2025

.

Email dialog

Paul Borrill: On Mar 6, 2025, at 3:02 PM, Paul Borrill <paul.borrill@icloud.com> wrote:

Robert,

Are you familiar with this paper?

A note on reliable full-duplex transmission over half-duplex links.

[1] LYNCH, W. C. Reliable full-duplex file transmission over halfduplex telephone lines. Comm. ACM 11, 6 (June 1968), 407- 410.

[2] Bartlett, K. A. Transmission control in a local data network. Proc. IFIP Cong. 1968 (to be published).

Kind regards, Paul

Robert Garner: On 3/7/2025 9:24 PM, Robert Garner wrote:

Paul,

Thanks for forwarding the 1968/69 note by Bartlett, Scantlebury and Wilkinson (attached below) with a comment section by Bill Lynch about his student Grant Saviers discovering the bug in the 1-bit version of the protocol and co-inventing the 2-bit version that Bartlett et al also invented.

Back in January, Bill Lynch (cc'd) had let me know of the issue with the 1-bit scheme, but I didn't get around to sending you the feedback, my apologies. I'm glad you nevertheless have discovered the issue. (I had seen a reference to the Bartlett note last year, but I hadn't gotten around to checking it out.)

Interestingly, Bill Lynch (cc'd) and Grant Saviers (cc'd, long-time CHM board member and supporter of the CHM's IBM 1401 restoration and demo project) are the only two people I personally know who programmed a UNIVAC I (at Case Western)!

Cheers,

– Robert

p.s. I wanted to ask you what you (and/or your Æthernet team members working on formulating a "new Open Atomic Ethernet protocol for Chiplet interconnects within Racks") think of Nvidia's Spectrum-X "lossless" Ethernet system? (I had thought Nvidia was wed to Infiniband via its Mellanox acquisition): https://www.naddod.com/blog/spectrum-x-nvidia-s-answer-to-gen-ai-ethernet-challenges https://resources.nvidia.com/en-us-accelerated-networking-resource-library/nvidia-spectrum-x https://nvdam.widen.net/s/56htdjbljg/ethernet-switches-datasheet-new-spectrum-x-data-sheet-2761336

Using RDMA over Converged Ethernet (RoCE), Spectrum-X Ethernet features:

> "RoCE Adaptive Routing avoids congestion by dynamically routing large AI flows away from congestion points. This approach improves network resource utilization, leaf/spine efficiency, and performance. The Spectrum-4 switch employ fine-grained load balancing, re-routing active flows to eliminate congestion. Additionally, the BlueField-3 DPUs work in tandem to handle out-of-order packets, placing packets in the correct order in the destination memory," ... "elevating bandwidth utilization from the typical 50-60% to over 97%," shunning Priority Flow Control (PFC) and Data Center Quantized Congestion Control (DCQCN).

> "RoCE Congestion Control collects network performance data with in-band network telemetry. The BlueField-3 DPUs use the collected switch telemetry data to optimize network data rates. BlueField algorithms use deep learning models for data metering, optimizing settings for multi-job, multi-tenant systems."

Nvidia claims it's "fully standards-based Ethernet, completely interoperable with Ethernet-based stacks" but it requires a "license that enables the software-defined, hardware-accelerated feature set required for generative AI on Ethernet."

"Traditional cloud services generate thousands of flows that are randomly connected to clients around the globe, which gives the cloud service network high entropy. However, AI and storage workloads tend to generate very few, but very large flows. These large AI flows dominate the bandwidth usage per link, significantly reducing the total number of flows and resulting in very low entropy for the network. This low entropy traffic pattern, also known as an elephant flow distribution, is typical with AI and high-performance storage workloads."

"Many flows suffer from high completion times because static ECMP hashing is not bandwidth-aware so some ports on the switch are highly congested while some others are underutilized. As flows finish transmission and release some port bandwidth, the lagging flows pass through the same congested ports, causing more congestion. This is because forwarding is static after the header has been hashed."

"For every packet forwarded to an ECMP group, the switch selects the port with the minimal load over its egress queue. The queues that are evaluated are those that match the packet's traffic class. As different packets of the same flow travel through different paths of the network, they may arrive out of order to their destination. At the RoCE transport layer, the Spectrum-X SuperNIC takes care of the out-of-order packets and forwards the data to the application in order. This renders the magic of RoCE Adaptive Routing invisible to the application benefiting from it."

"Spectrum-X switches employ packet spraying, selecting the port for every network packet with the minimal load over its egress queue. Spectrum-X switches also receive status notifications from neighboring switches, which can influence the forwarding decision. The queues evaluated match with the appropriate traffic class. As a result, Spectrum-X enables up to 95 percent effective bandwidth across the hyperscale system at load, and at scale."

The Nvidia Spectrum-X SN5600 Ethernet Switch features "64 x 800G OSFP ports, with flexible expansion to 128 x 400G ports or 256 x 200G ports" and their BlueField-3 SuperNIC offers "up to 400Gbps RDMA connections over RoCE." "The Spectrum-4 switch delivers the industry's lowest-latency 200/400GbE switching, ensuring ultra-low latency and jitter for 256 ports." The Spectrum-4 Ethernet chip "has total bandwidth of 51.2TB/s and includes 100 billion transistors, measures 90x90mm, has 800 solder balls on the bottom of the chip package, and consumes 500W of power." The entire switch consumes 2,800 watts.

"LinkX cabling series focuses on 800G and 400G end-to-end high-speed connectivity, leveraging 100G PAM4 technology and fully supporting the OSFP and QSFP112 MSA standards. It covers a range of cable types from DAC, ACC to multi-mode and single-mode optical modules, addressing diverse cabling needs. Specifically, these cables can seamlessly interface with the 800G OSFP ports on the SN5600 switch, enabling 1-to-2 split 400G OSFP port expansion, improving network flexibility and efficiency."

Comparing InfiniBand to Spectrum-X Ethernet from Comparing InfiniBand to Spectrum-X Ethernet

"Given how good InfiniBand is, why is NVIDIA also working on Ethernet? Intuitively, Ethernet's market foundation, versatility, and flexibility should be significant factors. Jensen Huang mentioned in his keynote speech that "we want to bring generative AI to every data center," which requires forward compatibility. "Many enterprises deploy Ethernet," and "it is difficult for them to acquire InfiniBand capabilities, so we bring such capabilities to the Ethernet market."

Flow-control issues with InfiniBand: "[InfiniBand's credit-based] mechanism also has many problems. For example, if a sending node sends data to a receiving node at a faster rate than the receiving node can process it, the receiving node's buffer may become filled, and the receiving node may be unable to return credits to the sending node, causing the sending node to be unable to send more data packets due to credit exhaustion. In addition, if the receiving node is unable to return credits and the sending node is also a receiving node for other nodes, the problem of backpressure spreading to a larger range may occur under bandwidth overload. Other problems include deadlock and error rate issues caused by different components."

"[InifiniBand's] credit flow control mechanism is difficult to respond quickly to changes in the network environment. If there is a large amount of diverse traffic in the network, the receiving node's buffer state will change rapidly. If the network becomes congested, the sending node is still processing earlier credit information, making the problem more complex. Additionally, if the sending node is constantly waiting for credits and switching between the credit and data transmission states, it can easily cause performance instability."

Bill Lynch: On Mar 8, 2025, at 4:40 PM, Bill Lynch wrote:

Well, I had no memory of the Bartlett paper or my reply until you brought it to my attention.

I'd like to reinforce some of the Bartlett comments. Optimizing these algorithms certainly does depend on what error model is being used. The whole impetus for my investigation was the fact that the performance of the link was dominated by strong, sporadic, fractional second, switching noise coupled into a POTS that neither God nor man intended to carry data. (The usual assumption of Gaussian noise wasn't adequate.) This noise burst often lasted through a packet transmission and then the transmission of the ACK/NAK from the previous transmission and blew away any independence assumption. I don't expect to see this kind of noise in today's comm lines.

Secondly, I often thought of the alternation bit as a mod 2 counter.s It represented the mod 2 function applied to ordinal number of the input packet. This point of view can be usefully extended to the mod k function applied to the ordinal of the input, giving an "alternation" field of $log_2(k)$ bits or so.. This would allow the received state of up to the last k packets to be known and provide a correction basis for a multi-hop pipeline of links up to K in length (think TCP here). Among other things it shows how to validate the Bartlett conjecture that a scheme could be constructed that would only fail with too many (more than K in this case) sequential packet errors! Think dead-man switch territory.

These kind of problems and situations are NOT obsolete. They are the result of a fundamental limit on the velocity of signals and the resulting relativistic situations. Agents in these kinds of communication systems can never know the current state of other agents - only perhaps what the state of other agents used to be. There were many similar situations with the coax Ethernet. I await with interest to see if the situation will be different in the quantum world.

Regards, Bill

Dear Robert,

We appreciate your detailed engagement and your thoughtful references – Wozencraft & Holstein's paper is a fantastic historical touchstone for this discussion. You are raising exactly the kinds of questions that help sharpen our work, so let me clarify some aspects of Æthernet that may not have been fully articulated yet in this forum.

AEthernet and the Fundamental Problem: Why L2 Needs Reliability

Robert if you can make perfect the L2 data-link protocol layer, application software will still need to confront and work around all the same issues: emergent metastable failure/"death spirals" in nodes, failed or intermittent faulty switches and nodes ("stragglers", "fail-slow"), application-level deadlocks and livelocks, Byzantine failures, an on and on.

We fully agree that application software must handle a wide range of failures—metastability, node crashes, Byzantine faults, and human-initiated failures among them. However, the specific network-induced failures we are addressing are distinct and pervasive in all datacenters, including those on the edge:

Application-level workarounds for network partitions add unnecessary complexity and cost. Æthernet eliminates the need for energy consuming retry mechanisms in higher level protocols and applications by providing atomic guarantees at layer 2, so applications are not forced to detect network faults themselves and then perform compensating transactions. see Scope from our original Æthernet proposal:

> Align what the network does with what the distributed application needs to accomplish. Build a world where the network is not allowed to create a mess the application must discover and then clean up, but rather must either ensure both sender and receiver (user space software endpoints in the distributed application) know a message was successfully sent and received, or only the sender "instantly" knows it failed, and no other node or part of the network knows that message ever existed.
>
> This entanglement between sender and receiver, with no library, software, or hardware permitted to disturb that entanglement, is a fundamental change from the "throw it over the wall and hope it gets there" of the last 50 years. From a different perspective, a network message becomes a transaction which exhibits ACID properties: atomicity, isolation, and durability directly, and consistency because that message exists only between endpoints and is not visible to any other observer.

Network-induced metastability is different from software-driven metastability. You correctly note that many failures in the Twitter study were due to human reconfiguration and garbage collection. However, network-layer metastable failures arise due to network-induced out-of-order or inconsistent state updates—an issue exacerbated by the forward-in-time-only (FITO) assumption. By guaranteeing in-order atomicity at the network level, we eliminate a class of failure states that today's networks expose to distributed systems.

Æthernet does not prevent all metastable failures—but it eliminates the ones caused by packet misordering, retries, and partial network partitions. These are some of the most pernicious failure modes, as they create invisible, undetectable (at time of failure) inconsistencies that only surface much later at the application layer.

2. The ACK/NAK Overhead: What's Different in AEthernet?

Your throughput calculations based on Metcalfe's ARPANET-era model are a fair starting point, but they assume a fully serialized, per-packet ACK scheme, which AEthernet does not use. Instead, AEthernet implements:

Aggregate Acknowledgments– Multiple packets are acknowledged together.

Distributed Cascade Commit Protocol (DCCP) for consensus and other cluster operations – Rather than traditional per-packet ACKs, we use an atomic commit mechanism similar in principle to distributed transaction commits, eliminating the "throttling" effect of naive ACK/NAK protocols.

Parallel acknowledgment paths– AEthernet supports a side-band acknowledgment path similar to the ALOHA two-channel model, ensuring acknowledgments do not consume primary link bandwidth.

Robert Garner: [Robert] What percentage of link traffic is consumed by status messages during fault-free operation?"

In Æthernet, status messages account for less than 1% of bandwidth in normal operation because they are sent in aggregated bursts rather than per packet. We also note that bandwidth is not the scarce resource in distributed microservices, high frequency trading, or AI Inference. Latency is. In fault conditions, the overhead increases only in proportion to actual faults, avoiding unnecessary signaling (and energy dissipation) when links are healthy.

Your ALOHA analogy is interesting—it's true that unregulated ALOHA networks could spiral into congestion collapse. AEthernet avoids this through strict backpressure mechanisms via dynamic allocations, ensuring nodes never send beyond their allocated bandwidth without a deterministic acknowledgment.

Deterministic Failure Detection Without TAR (Timeout And Retry)

Robert What if the node(s) performing round-robin checking go down or are inaccessible?"

Æthernet does not rely on a single node for failure detection. Instead, failure detection is distributed across multiple nodes using Local-Observer-View (LOV) information. The key distinction is:

Failure detection in Æthernet is purely local–each node only needs to observe the state of its immediate neighbors, unlike traditional heartbeat-based cluster-wide detection. Æthernet employs event-driven failure notification instead of periodic polling—eliminating many of the latency and scalability issues of heartbeat-based systems.

The lack of events on one link, compared to the pace of events on another, can trigger a local node-check where a working neighbor checks with surrounding neighbors before removing the node from routing tables. Routing tables are hop-by-hop so the sender and receiver don't care if the path has a kink, though every failover has a predictable effect on end to end latency. Unlike TCP retransmission-based recovery, Æthernet ensures real-time lossless detection and failover without requiring application-layer compensations.

In the case of rerouting packets to mask partial partitions — We reframe this issue by using atomic forwarding commitments to ensure that every transmitted packet is either fully delivered or fully rejected, avoiding partial delivery failures. This prevents intermediate congestion by coordinating sender-receiver state at the network level rather than relying on hop-by-hop best-effort delivery.

This is a direct countermeasure to partial partitions, where conventional networks leave application-layer protocols to detect and recover from network inconsistencies.

Must Latency and Bandwidth Trade Off?

You've challenged the notion that higher bandwidth doesn't necessarily hinder latency—and you're right. However, conventional Ethernet introduces latency through inconsistent delivery timing, which applications must compensate for. And for applications that demand latency consistency, the solution is to invest into extremely proprietary rollouts of networks and hardware. Æthernet flattens the latency distribution by enforcing strict ordering and atomicity guarantees at L2, which reduces worst-case latency rather than just improving average-case latency.

Your alternative title—"To ACK or Not to ACK: A Rethink of Ethernet's Data Link Layer"—is an interesting proposal, but we would modify it slightly to:

"To ACK or Not to ACK: Ethernet's Missing Atomic Commit Layer."

Why Æthernet is Positioned at L2, Not L4

Robert Garner: [Robert] This all sounds good. L4 is the designated level for reliable transport protocols. You're yet to convinced me that it should be at the L2 Data Link layer.

The problem is reliability cannot be recreated at L3+.

L4 reliability (e.g., TCP) is endpoint-based, whereas Æthernet operates hop-by-hop. This means failures are detected and recovered at each network segment rather than requiring end-to-end retransmissions.

At L4, reliability mechanisms like Homa and paired RPC protocols still assume a lossy L2. Æthernet eliminates this assumption and prevents reliability issues before they propagate to upper layers.

Applications running on Æthernet require less complexity. They no longer need elaborate failure recovery and redundancy architectures because the network itself guarantees atomicity and ordered delivery.

The real question isn't whether Æthernet replaces TCP, but whether an atomic and deterministic network layer can reduce the burden of reliability in L3+.

Conclusion (so far)

Ethernet (and all unreliable communication networks) suffer from an extreme and highly painful weakness: the failure of a single link can introduce unpredictable and disruptive failures across the entire network. "But links don't fail that often" you may hear from networking folks. They may grudgingly agree that it happens more often at scale in hyperscale datacenters. This is an illusion. Failures occur at all scales, and it always comes down to detecting the flakiness of each individual link.

The number of failure modes exposed to the distributed applications is three orders if magnitude worse than the simple one-link, one-path scenario in your example calculation.

The purpose of this insight is to show how to incorporate the classical equivalent of a Schrödinger cat into each Æthernet Link using a simple pair of back-to-back Shannon channels; turning the problem of link failure detection upside down and detecting "Entanglement loss" in a simple triangle relationship with local (near neighbor) connections.

This Entanglement protocol is a dance between two entities: It takes two to tango, but it takes 3 to party.

We now understand this better, and can use non-quantum concepts and terminology thanks to Bill and Grant.

Next Steps

1. Sahas and I are refining the specification for Æthernet's acknowledgment model and failure detection approach, and we'd love to incorporate your feedback into the next draft.
2. If you're open to it, we'd be happy to walk through a whiteboard session at our place in Saratoga to clarify any remaining concerns?

Looking forward to hearing your thoughts.

Kind regards, Paul & Sahas

On Mar 15, 2025, at 5:46 PM, Robert Garner wrote:

Paul,

Thanks for elaborating on AEthernet's rationale, goals and high-level characteristics.

Clarifying the Fundamental Problem

The key problem we are addressing is the inherent fragility of existing Ethernet-based networking stacks in distributed systems, especially when dealing with partition tolerance, latency, and exactly-once semantics. This fragility is exposed in: Lossy networks forcing upper-layer retransmissions → leading to unbounded tail latency.

Inability to provide strong consistency guarantees → forcing applications to handle failures, increasing complexity. Silent data corruption due to retries and partial failures → leading to undetectable integrity violations.

I'm not sure if you're hearing my point that even if you can make perfect the L2 data-link protocol layer, application software will still need to confront and work around all the same issues: emergent metastable failure/"death spirals" in nodes, failed or intermittent faulty switches and nodes ("stragglers", "fail-slow"), application-level deadlocks and livelocks, Byzantine failures, an on and on. Although we don't yet have a complete AEthernet design proposal, I don't see how it would address/ameliorate the latent bugs and design flaws in the complex and intricate distributed applications themselves (typically triggered by unexpected node hardware failure or faults or human initiated system reconfiguration actions, not lost, out-of-order, or duplicate network packets).

On the Alleged Throttling Due to ACK/NAK

Paul Borrill: AEthernet is not a naive stop-and-wait protocol.

Robert Garner: Thanks for clarifying. I did not see that from the draft presentation you shared on the 12th or from previous emails. But it's nice to know that Metcalfe's ARPAnet-era equation for calculating throughput is still applicable to 100 Gb/s links. (Btw, a 64-byte, 100-Gb/s packet would fit within 1/20th of a single bit of the original 10Mbps Ethernet! :)

From the 3/12 presentation, AEthernet appears to define a set of link status messages called "protocol," "state machine transition," and "link liveness," for up to 256 link state machines. (Fyi, the original PARC Ethernet packet format supported 256 addresses. :)

<PastedGraphic-1.png>

<PastedGraphic-2.png> <PastedGraphic-3.png>

Question —> Is there a sense yet of the percentage of link traffic consumed by these status messages (during say typical fault-free operation)? Then we can calculate a multiplexing factor. Or are ACK messages returned in a separate parallel link? (Like in the original two-channel ALOHA system :).

Deterministic failure detection without Timeouts And Retry (TAR)

I believe you mentioned round-robin checking of liveness heartbeats?

Question → What if the node(s) performing the round-robin checking go down or are inaccessible?

Atomic Commit Transactions (ACT) to eliminate retry storms, reconstruction storms and Metastable Failures. To make sure I understand this rationale: Are you suggesting that the AEthernet's Atomic Commit Transactions (ACT) could address node metastable failure states?

Thanks for the pointer to the 2022 paper "Metastable Failures in the Wild" from Twitter. It defines a metastable failure state that "keeps the system in the metastable failure state even after the trigger is removed." (That's quite a compelling name for it: "death spirals!" [Sean Lynch, 2021]) "Bronson et al. define the metastable failure state as the state of a permanent overload with an ultra-low goodput (throughput of useful work). They also define the stable state as the state when a system experiences a low enough load than it can successfully recover from temporary overloads, and the vulnerable state as the state when a system experiences a high load, but it can successfully handle that load in the absence of temporary overloads. A system experiences a metastable failure when it is in a vulnerable state and a trigger causes a temporary overload that sets off a sustaining effect—a work amplification due to a common-case optimization—that tips the system into a metastable failure state. The distinguishing characteristic of a metastable failure is that the sustaining effect keeps the system in the metastable failure state even after the trigger is removed.

Reading the paper, half of the metastable failures/death spirals were caused by engineers reconfiguring the systems! Only one was caused by a "network disruption." In the paper's featured examples, the authors didn't proffer any silver bullets, but it seems that for some a backoff algorithm would help. The analyzed cases were: 1) Twitter's garbage collection architecture, 2) NoSQL's replicated state machines, and 3) a look-aside cache scheme. For the 1st, no ideal fix was identified: "larger memory sizes decrease the memory pressure, which lowers the effect of GC. Thus, the system is less vulnerable with more memory and can sustain higher trigger durations and higher requests per second." For the 2nd: "timely removal of the trigger can prevent the transition into the metastable failure state." And for the 3rd: "there is a trade-off with setting the request timeout— a higher timeout decreases the vulnerability, but it takes longer to detect failed requests, whereas a lower timeout can quickly detect issues, but increases the metastable vulnerability." Note that none of these examples have much to do with network problems.

Btw, I found that the description of the metastable failure state analogous to when a pure ALOHA network becomes locked up in an exponentially declining performance regime when triggered/pushed over its capacity threshold (1/2e load) by users/nodes that don't back off/shed load. This degenerate phenomenon led to many academic studies of broadcast ALOHA and presumed Ethernet network instability — and to Boggs' retort that "Ethernet works in practice, but allegedly not in theory." (In other words, "there might be theoretical corner cases that cause the network to lock up, but I didn't see them in my heavily loaded performance study at DECSRC." :)

This is not TCP-style windowing or simple sub-stream multiplexing but a fundamentally new way to ensure liveness and atomicity without requiring application-layer compensations.

By addressing hotspots with Local-Observer-View (LOV) information only at L2 and below, we can guarantee that buffers at the destination never overflow.

AEthernet rethinks this with in-order atomic packet commitments, Local-only information failure detection and healing — far faster than any protocol stack or application can perceive, to enable network-coordinated transactions.

I look forward to the low-level details.

By solving these fundamental atomicity and in-order delivery problems at (or below) L2, everything above remains compatible with the applications that use them. Nevertheless, the applications (running on nodes) can still suffer from all the degradation problems you've brought up.

We also plan to provide extremely simple primitives to support membership management on behalf of distributed applications. This is hinted at in the design of our logo. AEthernet also integrates with features at higher protocol levels (above L2)?

show that traditional Ethernet makes partitions an application problem, whereas AEthernet aims to make the network itself provably resilient. Except perhaps when it's being reconfigured by humans? (per the Twitter's metastability paper)

Nifty and other user-space workarounds are band-aids. AEthernet proposes a foundational change: network-level atomicity guarantees that prevent these failure cascades in the first place. Question —> According to the paper, would the AEthernet L2 protocol "reroute packets between end hosts to mask partial partitions" (say due to a switch dying) since otherwise there would be an "increased load on the intermediate nodes and performance bottlenecks"? If so, how would AEthernet "expose the network state to the system running atop of it to facilitate building system-specific optimizations"? This doesn't seem practicable. An analogy(?): In order to recover from a broken/down mail distribution center, the mail handlers would need to send letters to all the mail system's users (based on the envelope's return addresses) instructing them to reduce their mail load until the distribution center is repaired? The Nifty algorithm may "in theory be a band-aid, but can work in practice?" :)

John Ousterhout believes that TCP (L3) needs to be replaced, and proposes Homa: Larry Peterson thinks It's Groundhog Day on the debate in TCP, and proposes using 'paired' (request- response RPC protocols) instead of TCP. This all sounds good. L4 is the designated level for reliable transport protocols. You're yet to convinced me that it should be at the L2 Data Link layer.

Factions Fragmenting Ethernet These approaches drive Ethernet toward vendor-controlled, closed ecosystems. I agree this is problematical. However, it seems that these huge cloud firms now have the wherewithal to define and pay for their own networking standards, chips and hardware. It's astounding (but perhaps not unreasonable) that all of them are implementing their own full-custom chips (much to Intel and AMDs chagrin and TSMC's take.)

Your suggestion for a title revision—"To ACK or Not to ACK?"—is an interesting framing, but we would rephrase it as: "Must Latency and Bandwidth Trade Off? A Rethink of Ethernet's Reliability Model." I still don't see how this is a latency vs. bandwidth question. Are you hearing my position that greater bandwidth also reduces the latency for application level message and data streams? Higher packet throughput on Ethernet isn't hindering latency.

The real issue is not whether ACKs should exist but how they should be implemented to preserve both efficiency and correctness. Also, at which network layer?! How do you convince readers that the L2 Data Link layer is where ACKs should live? How about this for a title: "To ACK or Not to ACK: A Rethink of Ethernet's Data Link Layer" ?

I welcome further engagement and critical analysis as we refine the standard. Thanks for being open to and responding to my what-must-seem ill formed and stiff questions. My excuse is that I would like to see more detailed, low-level information.

Sincerely,

— Robert

p.s. I found an even earlier reference to an ACK-based protocol in Will Crowther's 1971 paper "Flow Control in a Resource Sharing Computer Network," (where Will described ARPAnet's initial single-outstanding "naive stop-and-wait ACK protocol"). It's a 1961 information theoretic paper "Coding for Two-Way Channels" by Wozencraft and Holstein, attached here. I can't say I fully grok it. :) Coding for two-way channels

On Mar 15, 2025, at 6:49 AM, Paul Borrill wrote:

Dear Robert, Bill, and colleagues,

Thank you for your detailed responses and insightful engagement in this discussion. It is clear that we are all aligned in seeking improvements to Ethernet's reliability, efficiency, and openness. However, I would like to clarify some key points and address some misconceptions regarding AEthernet.

I've added Alan Karp, who wrote Rust and Javascript simulators to explore these issues. And Bijan Nowroozi, who is giving a talk this coming Wednesday in the Open Atomic Ethernet forum on his perspectives.

Clarifying the Fundamental Problem

The key problem we are addressing is the inherent fragility of existing Ethernet-based networking stacks in distributed systems, especially when dealing with partition tolerance, latency, and exactly-once semantics. This fragility is exposed in:

Lossy networks forcing upper-layer retransmissions → leading to unbounded tail latency. Inability to provide strong consistency guarantees → forcing applications to handle failures, increasing complexity. Silent data corruption due to retries and partial failures → leading to undetectable integrity violations.

The research into partial network partitions has demonstrated that current assumptions in distributed systems design (i.e., that networks are either "up" or "down") fail in real-world scenarios. AEthernet is designed to fundamentally rethink this model.

On the Alleged Throttling Due to ACK/NAK

Robert, your calculations assume a traditional per-packet ACK model, but AEthernet is not a naive stop-and-wait protocol. We incorporate:

- Aggregate acknowledgments across multiple packets and multiple links.
- Deterministic failure detection without Timeouts And Retry (TAR)
- Atomic Commit Transactions (ACT) to eliminate retry storms, reconstruction storms and Metastable Failures.

This is not TCP-style windowing or simple sub-stream multiplexing but a fundamentally new way to ensure liveness and atomicity without requiring application-layer compensations.

Your calculation using Metcalfe's model assumes that each 64-byte packet requires an immediate round-trip ACK before sending another. AEthernet's pipelined, distributed acknowledgment scheme does not suffer from this limitation. Instead, acknowledgments cascade asynchronously across a structured transaction layer, mitigating the assumed bandwidth collapse.

By addressing hotspots with Local-Observer-View (LOV) information only at L2 and below, we can guarantee that buffers at the destination never overflow. Instead, we push the problem back to the senders, who remain responsible for the resource usage (buffer allocations) for their application. Our protocol goes beyond credit-based-flow-control, and guarantees "symmetric interactions" (needed by transactions), not flows (needed by video). It's a different way of thinking about things, and we can prove that it works. As the Ethernet community is fond of saying: "It doesn't work in theory, it only works in practice".

While we can eliminate bottlenecks at the destination, we 'avoid' the undesirable behavior of switches which cause retry storms, by not going through switches for the main fabrix. The L3 boundary is at the Top of Rack Switch (TOR). Our mesh can be completely 'scale independent' inside this boundary, although in our market positioning to stay within a rack for our microdatacenters on the edge. Of course, if the switch vendors adopted our protocol, they can also benefit from this breakthrough. But we don't require them to do that, any more than VMWare needed Microsoft's permission to revolutionize the operating systems world.

John Ousterhout believes that TCP (L3) needs to be replaced, and proposes Homa:

"In spite of its long and successful history, TCP is a poor transport protocol for modern datacenters. Every significant element of TCP, from its stream orientation to its expectation of in-order packet delivery, is wrong for the datacenter. It is time to recognize that TCP's problems are too fundamental and interrelated to be fixed; the only way to harness the full performance potential of modern networks is to introduce a new transport protocol into the datacenter. Homa demonstrates that it is possible to create a transport protocol that avoids all of TCP's problems. Although Homa is not API-compatible with TCP, it should be possible to bring it into widespread usage by integrating it with RPC frameworks."

Larry Peterson thinks It's Groundhog Day on the debate in TCP, and proposes using 'paired' (request- response RPC protocols) instead of TCP.

From our perspective, both John and Larry are correct: Datacenters (not just the Internet) seems to be "missing a standard protocol for the request/response paradigm". We believe the problem goes a little deeper, and we need to enhance L2 as well, because it's easier to make heartbeats scaleable and hidden across a single link than it is across an entire Clos network. This ties in directly to our intuition about Quantum Information Theory, and Multiway Systems. The magic of making this work at scale a mathematical technique called 'compositionality'. If it's provably correct and formally verified on a single link, only then can it be "composed" across many links and still maintain its guarantees.

Sequencing numbers are a 'Forward in Time Only' (FITO) concept. Our protocols include the counterfactual4 In our protocol, sequence numbers can go backwards), and incorporate a finite partial order set (in both directions) to manage the operations on data structures, to guarantee they will not be corrupted, provided both the forward and reverse sequencing is 'successive'. Monotonicity is neither necessary nor sufficient. The concept of monotonicity keeps us stuck in the FITO world, and prevents us seeing the more general solution.

Larry Peterson provides a nice description of the request/response paradigm (as an alternative to TCP) and how it relates to 'exactly once' semantics in Chapter 5.3 of his book Computer Networks: A Systems Approach:

"RPC reliability may include the property known as at-most-once semantics. This means that for every request message that the client sends, at most one copy of that message is delivered to the server. Each time the client calls a remote procedure, that procedure is invoked at most one time on the server machine. We say "at most once" rather than "exactly once" because it is always possible that either the network or the server machine has failed, making it impossible to deliver even one copy of the request message."

By solving these fundamental atomicity and in-order delivery problems at (or below) L2, everything above remains compatible with the applications that use them.

We also plan to provide extremely simple primitives to support membership management on behalf of distributed applications. This is hinted at in the design of our logo.

3. Partial Network Partitioning and Data Corruption

You argue that "trusted transactional applications" handle faults well, but the empirical evidence from cloud-scale systems contradicts this. Large-scale failures (e.g., in Google Spanner, Amazon DynamoDB, and Azure Cosmos DB) show that traditional Ethernet makes partitions an application problem, whereas AEthernet aims to make the network itself provably resilient.

The 2023 study on partial network partitions highlights:

- 80% of failures result in catastrophic data loss (not just service degradation).
- 90% of failures are silent—undetected by existing protocols.
- 21% of failures cause permanent, irrecoverable damage—even after partitions heal.

Nifty and other user-space workarounds are band-aids. AEthernet proposes a foundational change: network-level atomicity guarantees that prevent these failure cascades in the first place.

4.Factions Fragmenting Ethernet

You asked which factions are driving fragmentation. The key players introducing proprietary, closed networking models are:

NVIDIA (Spectrum X, BlueField DPU) → pushing for end-to-end CUDA-accelerated networking. Broadcom (Jericho, Trident series) → optimizing for vendor lock-in via hyperscaler-specific silicon. Intel (IPU/Fabric, used extensively in Google's infrastructure) → attempting to redefine Ethernet as a NUMA-style interconnect. Meta's Arista partnership → promoting a proprietary hyperscale switching fabric.

These approaches drive Ethernet toward vendor-controlled, closed ecosystems. AEthernet is explicitly not a proprietary protocol; it is an open, community-driven standardization effort.

5. The Real Debate: "To ACK or Not to ACK?"

Your suggestion for a title revision—"To ACK or Not to ACK?"—is an interesting framing, but we would rephrase it as:

"Must Latency and Bandwidth Trade Off? A Rethink of Ethernet's Reliability Model."

The real issue is not whether ACKs should exist but how they should be implemented to preserve both efficiency and correctness. AEthernet rethinks this with in-order atomic packet commitments, Local-only information failure detection and healing — far faster than any protocol stack or application can perceive, to enable network-coordinated transactions.

Next Steps:

Sahas and I will integrate this discussion into our revised paper, addressing both the theoretical underpinnings and practical performance characteristics of AEthernet. I welcome further engagement and critical analysis as we refine the standard.

Looking forward to your thoughts.

Kind regards, Paul

On Mar 14, 2025, at 11:14 PM, Robert Garner <robgarn@mac.com> wrote:

Paul,

Thanks for following up on my inquiry about rationale for your “Open Atomic Ethernet” (AEthernet/AE) project: [Borrill] The [Ethernet] industry is being fragmented now by several factions which are trying to establish proprietary control. [Garner] Can you elaborate on who are the factions? Does this include nVidia’s Spectrum X Ethernet? Who else?

Is AE the work of a particular firm or of an (open) band of concerned folk?

The claim that bandwidth collapses with ACK/NAK is the narrative Broadcom and nVIDIA would like everyone to believe. Although I’ve used the adjective “throttled” instead of the melodramatic “collapse,” unless the network supports the ACK’ing of multiple outstanding packets, I concur that ACKs at the lowest level of the network can reduce the peak available throughput and increase latency for larger application-level messages/data transfers. (Also note that, over the years, a more complex ACK’ing architecture would need to be maintained across network generations and speeds.) Reliable delivery should/can be handled by the upper protocol and application layers! (see the great example below for handling partitioned networks.)

Maintaining liveness and synchronizing processes in networks is a challenge when packets can be dropped, reordered, duplicated or delayed Because of hardware faults and failures, firmware bugs, deadlocks, resources limitations, etc, these events will never cease/go away.

This makes exactly-once semantics impossible and precipitates retry storms which lead to unbounded tail latency and transaction failure It’s been true for a long time that robust/trusted transactional applications have been able to handle all nature of faults and failures, including those that aren’t necessarily due to network failures, faults, or partitions.

This results in silent corruption of data structures and loss of data. A problem seen in every distributed database for decades My experience with large distributed systems — while at IBM I personally studied many dissatisfied customer failure reports — is that such failures are primarily due to hardware/firmware faults that provoke bugs/design flaws in the application software in unanticipated, misunderstood or untested ways. Large software applications are so enormously complex/intricate that they will contain design flaws/bugs for the life of the product! And the applications are too large to ever be able to write diagnostic suites that cover the unimaginably large number of corner cases (since the application can be interrupted by a failure/fault at any arbitrary moment in time.)

Networks Cause Applications To Fail [Partial Network Partitioning (2022/3]

Scanning through the 2023 “Partial Network Partitioning" paper, I see that many of the partitions studied there were caused by faulty or failed network switches that uncovered “design flaws”/bugs in the application (see my comment above). Nevertheless, I’m happy to see that the authors posited an application-layer solution at a higher level (no need to attempt the impossible task of failure-proofing the underlying systems):

"Our findings motivate us to build the network partitioning fault tolerance layer (Nifty), a simple, generic, and transparent communication layer that can mask partial network partitions (Section 6). Nifty’s approach is simple; it monitors the connectivity in a cluster through all-to-all heart beating, and when it detects a partial partition, it detours the traffic around the partition through intermediate nodes. Nifty overcomes all the shortcomings present in the studied fault tolerance techniques.”

Sahas and I will respond to the remaining questions in our update to the Paper (now renamed “Is There Necessarily a Tradeoff between Latency and Bandwidth in Ethernet-like Networks?”).

[Q1] I s this really a debate/skirmish between network latency and throughput? (from the perspective of applications)

May I suggest an alternative name: “To ACK or not to ACK?” (Assuming ACKs are a key aspect of your "exactly-once semantics”?)

How about study the pros and cons of each. Don’t vilify one.

This is an “open” project right? Not the hidden agenda of a particular firm?

Cheers,

— Robert

On Mar 13, 2025, at 6:44 PM, Paul Borrill <paul@daedaelus.com> wrote:

Bill,

Thank you for your comments. This is why I believe your perspective is a profound breakthrough; overlooked by the industry for far too long.

I’ve renamed and truncated this thread to focus on the fundamental issue in networking today: Latency and Partition Tolerance in Distributed Systems.

Robert,

The claim that bandwidth collapses with ACK/NAK is the narrative Broadcom and nVIDIA would like everyone to believe.

THE PROBLEM:

Maintaining liveness and synchronizing processes in networks is a challenge when packets can be dropped, reordered, duplicated or delayed Conventional networks require protocol stacks and applications to use timeouts and retries to maintain liveness This makes exactly-once semantics impossible and precipitates retry storms which lead to unbounded tail latency and transaction failure This results in silent corruption of data structures and loss of data. A problem seen in every distributed database for decades These failures have not previously been understood because customers (under NDA) may not publish results that embarrass vendors

THE CONSEQUENCE:

Networks Cause Applications To Fail [Partial Network Partitioning (2022/3] Silent, Uncorrectable, Data Corruption: [Understanding Partial Network Partitioning (2020)] 80% of failures have a catastrophic impact with data loss being the most common (2790% of the failures are silent, the rest produce warnings that are unclear 21% of the failures lead to permanent damage to the system This damage persists even after the network partition heals

THE SOLUTION:

What Bill describes below, plus new abstractions and implementations we plan to standardize in Open Atomic Ethernet and (later) the IEEE.

Sahas and I will respond to the remaining questions in our update to the Paper (now renamed “Is There Necessarily a Tradeoff between Latency and Bandwidth in Ethernet-like Networks?”).

Kind regards, Paul

On Mar 13, 2025, at 12:42 PM, Bill Lynch <W-C-Lynch@comcast.net> wrote:

Robert,

A couple of comments:

1) As per the comments in my last message - It is quite possible to use a higher level protocol ON TOP of an ACK/Nak protocol to achieve throughput approaching the connection's inherent throughput capacity. Basically such a protocol implementation should:

a) De-multiplex on a round-robin basis, the input stream into several sub-streams.

b) Transmit all sub-streams independently and concurrently using separate instances of an ACK/NAK protocol. The actual ACK/NAK protocol may even vary from sub-stream instance to instance.

c) At the receiving end re-multiplex the sub-streams back into a received copy of the original .

Since the bandwidth capacity of the connection will be finite, there will be limit to the number of sub-streams that will yield an significant improvement. This approach is not new - it is fundamentally the same as the flow control in TCP. It is only a question of the size and representation of the system state.

I think it is quite untrue that ACK/NAK is a severe and fundamental limit on transmission efficiency.

2) I do believe that Ethernet's primary asset is the large and thriving ecosystem that allows a multitude of vendors to contribute to a customer system. That ecosystem was grown by a free, open, and detailed system of standards provided first by the involved vendors and later by the professional standards association. I can't tell you how many improvements proposals emerged from even within those vendors - proposals that improved the system performance by a few percent at the cost of compatibility.

A bit more subtle was the substantial effort (beginning with the BlueBook and through the professional standards) provided not only the nominal value of parameters but also carefully considered tolerances constructed and analyzed to assure system operation when the system had been constructed of modules provided by various vendors (an original 2 user Ethernet could have equipment from 7 different vendors). This original DIX demonstration at the 1981 NCC in Chicago's McCormick Place was aimed directly at this.

There were a number of commercial Ethernet.networking vendors with offered systems that worked all the time when composed of components from that vendor (and some but not all of the time when composed of components from mixed vendors). The vendors either found their addressed market constrained or were force to remove the network system from the product line.

Regards, Bill

On 3/12/2025 10:54 PM, Robert Garner wrote:

Paul,

Responding to your reply to Geoff:

Geoff	Ethernet has remained dominant because of its economy, reliability, embedded expertise and the small headroom for improvement.
Paul	We are trying to solve a fundamental problem that has existed for Distributed systems and databases for decades.

Can you clearly state what the "fundamental problem" is?

What Robert, Bill and Grant have provided is a profound breakthrough in this investigation. What is/was our "pronounced breakthrough"?

My position is that AE's ACK-based protocol (based on what I can ascertain) appears to hobble throughput — while still requiring an upper level protocol for reliability to recommence after failures, faults, deadlocks, etc.. Also you need the bandwidth to reduce the latency of delivering application level chunks of data (not just AE's small 64-byte packets.)

Perhaps someone there could check the calculation in my previous email (copied below) that suggests that AE will only achieve 2/5'th to 1/3'rd the available link bandwidth.*

The [Ethernet?] industry is being fragmented now by several factions which are trying to establish proprietary control.

Can you elaborate on who are the factions? Does this include nVidia's Spectrum X Ethernet? Who else?

Is AE the work of a particular firm or of an (open) band of concerned folk?

Best,

— Robert

* Here's my throughput calculation for an ACK-based AE point-to-point link, as I understand from today's draft presentation:

Per my 1/15/25 email on Metcalfe's thesis throughput equation for an ACK-based network (copied below), the throughput degradation factor is $1/(1+T/Tp)$, so minimal impact due to ACKs occurs when $T << Tp$ that is $transmitter_timeout << data_packet_length$

or, expanding (and assuming zero time for the transmitter to detect the non return of an ACK), minimal impact occurs when $(roundtrip_prop_time + ack_packet_length) << data_packet_length$.

If the $ack_packet_length = 0$, then it's $roundtrip_prop_time << data_packet_length$.

It looks like the AE proposal is for 64-byte packets on 100-Gb/s 1.6-meter point-to-point links.

With the link running @100 Gb/s, the $roundtrip_prop_time = 2 * 1.6m * 3ns/m = 8ns$.

If ACK packets are 8-byte flits, then $ack_packet_length = 8 * 8b/100Gb/s = 0.64ns$ so $T = transmitter_timeout = 8 + 0.64 = 8.64ns$, and the degradation factor due to ACKs would be

$1/(1 + T/Tp) = 1/(1 + 8.64/5.12) = 0.37$, You're only achieving 2/5'th the available bandwidth.

In reality, it will take time for the transmitter state machine to respond to the lack of a ACK and retransmit the packet. If recognition and retransmission takes say 2 nanoseconds, then the degradation factor due to the ACK protocol would be

$1/(1 + T/Tp) = 1/(1 + 10.64/5.12) = 0.42$. You're only achieving 1/3'rd of the available bandwidth.

FAQ-ÆThernet

[Q2] What are Imposition Networks?

[A2] Imposition networks impose packets on the network without regard to whether the endpoints can accept them or not, OR, the if the network is congested or not.

[Q3] Why are Imposition Networks a problem?

[A3] By sending more/ bigger packets, you are requiring the presence of an intermediate nodes that can capture, buffer and route packets on their way to a predetermined destination

[Q4] I n Backpressure Flow Control [1], Goyal et al point out that without immediate back pressure ...

> **Abstract:** Effective congestion control in a multi-tenant data center is be- coming increasingly challenging with rapidly increasing workload demand, ever faster links, small average transfer sizes, extremely bursty traffic, limited switch buffer capacity, and one-way protocols such as RDMA. Existing deployed algorithms, such as DCQCN, are still far from optimal in many plausible scenarios, particularly for tail latency. Many operators compensate by running their networks at low average utilization, dramatically increasing costs.
>
> In this paper, we argue that we have reached the practical limits of end-to-end congestion control. Instead, we propose a new clean slate design based on hop-by-hop per-flow flow control. We show that our approach achieves near optimal tail latency behavior even under challenging conditions such as high average link utilization and in-cast cross traffic. By contrast with prior hop-by-hop schemes, our main innovation is to show that per-flow flow control can be achieved with limited metadata and packet buffering. Further, we show that our approach generalizes well to cross-data center communication.

[A4] "To compensate for tail latency, propose a hop-by-hop per-flow flow control instead of E2E"

[Q5] What's wrong with a predetermined destination?

[A5] It requires God's-Eye-View knowledge, which exists only in the minds of the network designers/ administrators. Assumes $\exists$ in your belief set.

[Q6] What's wrong with RDMA?

[A6] RDMA is a stove pipe owned by nVidia.

[Q7] How does it compare to PCIe?

[A7] PCIe6 on CXL is the same as PCI6.

[Q8] How long can PCI or CXL cables be?

[A8] A meter or two. (266 Byte Flit on CXL)

Bibliography

[1] P. Goyal, P. Shah, N. K. Sharma, M. Alizadeh, and T. E. Anderson, "Backpressure flow control," in *Proceedings of the 2019 Workshop on Buffer Sizing*, ser. BS '19. New York, NY, USA: Association for Computing Machinery, Jan. 2020, pp. 1–3. [Online]. Available: https://dl.acm.org/doi/10.1145/3375235.3375239